

Document number	P0748R0
Date	2017-07-14
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Wording for Networking PDTS ballot comments on reentrancy

Introduction

This paper presents proposed changes in response to four National Body comments for PDTS 19216 from [N4643](#). The comments all relate to reentrancy rules.

Changes

NB comment 008 (GB-3)

Reentrancy and run/dispatch.

In 17.6.5.8 [reentrancy] the C++14 standard says:

"Except where explicitly specified in this standard, it is implementation-defined which functions in the Standard C++ library may be recursively reentered."

In the executor requirements, the intention is that the `dispatch()` function may be recursively reentered. A statement to this effect may need to be added to the requirements. (All `dispatch()` member functions provided by executors in the TS itself should then by implication allow reentrancy.)

Proposed change: Explicitly specify that dispatch functions can be recursively re-entered.

Accept the change.

NB comment 015 (GB-6)

Reentrancy and `use_service/make_service`.

The intention is that both `use_service` and `make_service` may make nested calls (from the Service constructor) to `use_service` or `make_service`. Obviously these nested calls will require a different Service template argument. I am uncertain if calling these function templates with different template arguments counts as recursive reentrance, but if it does then we may need to add a sentence explicitly specifying that this is permitted.

Proposed change: Decide if it's needed and add a suitable sentence.

Accept the change.

LWG noticed some underspecification in 13.7 [async.exec.ctx] which makes specifying this difficult. I have tried to address that.

NB comment 016 (GB-7)

run()/run_one() specification overly restrictive on users.

Both the run() and run_one() functions include the following statement:

"Must not be called from a thread that is currently calling a run function."

This restriction was originally added as a way to prevent users from entering a kind of "deadlock". This is because run() and run_one() can block until the io_context runs out of work. Since outstanding work includes currently executing function objects, if a function object makes a nested call to run()/run_one() that nested call could block forever as the work count can never reach zero.

However, it has been brought to Chris Kohlhoff's attention by users that there are valid use cases for making these nested calls. Deadlock can be avoided if some other condition will cause run()/run_one() to exit (e.g. an exception, explicit call to stop, run_one finished running a single function, etc). This condition can be known ahead of time by the user.

The existing implementation in asio does not make any beneficial use of this restriction.

Proposed change: Strike those sentences from both those places. Make it the responsibility of the user to avoid the conditions for deadlock.

Accept the change, adding notes warning about the potential for deadlock.

NB comment 017 (GB-8)

Reentrancy and run functions.

The intention is that the run functions may be recursively reentered. We may want add a sentence explicitly specifying this.

Proposed change: Explicitly specify that run functions can be recursively re-entered.

Accept the change.

Discussion

The [reentrancy] wording in the IS is underspecified (see [LWG 2414](#)) but there are some uses of functions in the Networking TS that should always work recursively, and some that definitely can't. Instead of leaving it implementation-defined, we can explicitly specify those cases.

Proposed Wording

Changes are relative to [N4656](#).

Modify 13.2.2 [async.reqmts.executor] paragraph 4 as shown:

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads. The member function dispatch may be recursively reentered.

Modify 13.7 [async.exec.ctx] as shown:

-2- Access to the services of an `execution_context` is via three function templates, `use_service<>`, `make_service<>` and `has_service<>`.

-3- In a call to `use_service<Service>()`, the type argument chooses a service, identified by `Service::key_type`, from a set of services in an execution context. If the service is not present in an the `execution_context`, an object of type `Service` is created and added to the

`execution_context`. A program can check if an `execution_context` implements a particular service with the function template `has_service<Service>()`.

-4- Service objects may be explicitly added to an `execution_context` using the function template `make_service<Service>()`. If the service is already present, `make_service` exits via an exception of type `service_already_exists`.

-5- Once a service reference is obtained from an `execution_context` object by calling `use_service`, that reference remains usable until a call to `destroy()`.

Add a new paragraph to the end of 13.7 [async.exec.ctx]:

-6- If a call to a specialization of `use_service` or `make_service` recursively calls another specialization of `use_service` or `make_service` which would choose the same service (identified by key type) from the same execution context, then the behavior is undefined. [Note: Nested calls to specializations for different service types are well-defined. --end note]

Modify 13.7.5 [async.exec.ctx.globals]:

-4- ~~Notes:~~ Remarks: The reference returned remains valid until a call to `destroy`.

[...]

-7- ~~Remarks:~~ Throws: `service_already_exists` if a corresponding service object of type `KeyService::key_type` is already present in the set.

-8- ~~Notes:~~ Remarks: The reference returned remains valid until a call to `destroy`.

Modify 14.2 [io_context.io_context] paragraph 4 as shown:

The `io_context` member functions `get_executor`, `stop`, and `stopped`, the run functions, and the `io_context::executor_type` copy constructors, member functions and comparison operators, do not introduce data races as a result of concurrent calls to those functions from different threads of execution. [Note: The `restart` member function is excluded from these thread safety requirements. --end note] The run functions may be recursively reentered.

Remove 14.2.1 [io_context.io_context.members] paragraph 4 as shown:

~~-4- Requires: Must not be called from a thread that is currently calling a run function.~~

Under 14.2.1 [io_context.io_context.members] after paragraph 6 insert a new paragraph as shown:

-?- [Note: Calling `run` from a thread that is currently calling a run function may introduce the potential for deadlock. It is the caller's responsibility to avoid such deadlocks. --end note]

Remove 14.2.1 [io_context.io_context.members] paragraph 10 as shown:

~~-10- Requires: Must not be called from a thread that is currently calling a run function.~~

Under 14.2.1 [io_context.io_context.members] after paragraph 14 insert a new paragraph as shown:

-?- [Note: Calling `run` one from a thread that is currently calling a run function may introduce the potential for deadlock. It is the caller's responsibility to avoid such deadlocks. --end note]

Acknowledgments

Thanks to Chris Kohlhoff for the wording for 016 (GB-7).